

EVA API Spec — v0.3 Research

Node

Stage 1 output — Architect Agent

Status: Mostly **CONFIRMED** — this is the strongest of the four documents, since it's transcribed directly from the running `api.py`.

Server

✓ Confirmed from `api.py`:

- `http.server.ThreadingHTTPServer`, custom `EVAHandler(BaseHTTPRequestHandler)`
- `server_version = "EVA/0.3"`
- Entrypoint: `python -m eva.api [port] [db_path]` — defaults: port `8000`, db `eva.db`
- Deliberately **not** FastAPI — sandbox had no network access to install packages. Handler bodies are written so migrating to FastAPI later is a matter of copying each body into a path function.
- All responses are JSON (`Content-Type: application/json`), errors return `{"error": "..."}`

- Generic exception handling: `KeyError` → 400
"missing required field: {e}"; any other
Exception → 500 {str(e)}
-

Endpoints

`POST /user/create`

✓ Confirmed

Request: { "name"?: string } (defaults to
"anonymous")

Response (201): { "id": string, "name": string
}

`POST /mission/create`

✓ Confirmed

Request:

```
{
  "user_id": string,      // required
  "objective": string,    // required
  "source_text"?: string,
  "sources"?: array,      // shape per
  frontend: [{origin, description, text}]
  "user_confirmed"?: boolean
}
```

Response (201): { "id": string, "status": string, "objective": string, "report": string|null }

Errors: 400 if user_id or objective missing

POST /task/create

✓ Confirmed

Request:

```
{
  "mission_id": string,    // required
  "agent": string,         // required –
                           must be valid AgentName
  "description": string,   // required
  "auth_level"? : string   // defaults to
                           USER_APPROVED
}
```

Response (201): { "id": string, "status": string }

POST /authorize

✓ Confirmed – routes through Sentinel

Request:

```
{
  "mission_id": string,    // required
  "agent": string,         // required
  "description": string,   // required
  "task_id"? : string,     // generated if
```

```
omitted
  "auth_level"?: string,
  "user_confirmed"?: boolean
}
```

Response (200): { "decision": string,
"auth_level": string, "reason": string }

POST /memory/store

✅ Confirmed – routes through Ghost

Request: { "mission_id": string, "statement":
string, "source_id"?: string }

Response (201): { "id": string,
"classification": string, "confidence":
number, "conflicts": string[] }

POST /tool/execute

✅ Confirmed to exist (referenced by frontend +
earlier code fragment `core.tools.list_tools()`);
full handler body not yet seen in the `api.py` copy on
file – the version transcribed here only shows

`do_POST` branches through `/memory/store`. ?

**Open: need to confirm whether `/tool/execute` is a
branch inside this same `do_POST`, or defined
elsewhere.**

Request (from frontend usage): { "agent": string,
"tool_name": string, "mission_id": string,
"params": object }

Response (from frontend usage): `{ "success":
boolean, "data": any }`

GET /tool/list

✓ Confirmed to exist: `self._send_json(200,
{ "tools": core.tools.list_tools() })`. Full route
context not in the `api.py` copy on file – same open
question as above.

GET /mission/status

✓ Confirmed

Query: `mission_id` (required – 400 if missing)

Response (200): shape of

`core.get_mission_status(mission_id)` – ? exact
fields unknown, passed through as-is; 404 if mission
not found

GET /memory/search

✓ Confirmed

Query: `q` (string, default `""`), `mission_id` (optional
filter)

Response (200): `{ "results": [{ "id",
"content", "classification", "confidence" }] }`

GET /audit/history

✓ Confirmed

Query: `mission_id` (optional – all events if omitted)

Response (200): `{ "events": [{ "timestamp",
"agent", "action", "result" }] }`

Open Questions

1. Full `/tool/execute` and `/tool/list` route bodies — only the response line for `/tool/list` has been directly seen; need the surrounding `do_GET / do_POST` context to confirm they live in `EVAHandler` as shown, or in a separate handler.
2. Exact shape of `core.get_mission_status()` return value.
3. CORS handling — frontend (`index.html`) calls the API cross-origin from a static file; `api.py` as transcribed has no visible CORS headers. Either CORS is added elsewhere (e.g. a `do_OPTIONS` handler not yet seen) or the frontend needs a same-origin proxy. **This must be resolved before Stage 5 deployment.**

This document is safe for Stage 2/5 to build against for all  endpoints. Do not scaffold `/tool/execute` internals from the frontend's assumed shape alone — confirm against the real handler first.